

My friend posted on their instagram story a screenshot of a simple TikTok DM along the lines of: “I am dying, here is my bitcoin wallet with loads of money, here is the login”. Being the college freshman fascinated by cybersecurity and reverse engineering, I immediately reached out and got both permission from my friend and the link that he had previously censored. This write-up details the steps I used to reverse engineer this simple attack, and my reasoning along the way.

My first step in reverse engineering this was spinning up my sandbox environment of choice, a REMnux VM. I attempted a simple curl of the link provided, but alas, that returned an error code. After doing a brief bit of research into how the website knew I was accessing from the command line rather than a browser, I used the extra DoH flag. The html file that ended up being returned was piped into final.html (I thought this might be the end, but I was so very wrong). The page was relatively simple, just a bunch of fake assets and a login button. The login button redirected to a new html page, piped into login.html. This is where it got very odd for me, as I’m somewhat new to this. I had to somehow curl the page that login redirected me to statically, while passing in a username and password. The original phishing message did contain a username and password, so using the command:

```
curl -X POST '[url ending with login.html]' -H 'Content-Type: application/x-www-form-urlencoded' -d 'phone_mob = btc998 & pwd = uu88778 & captcha = 1' -v
```

This did not work. I had assumed that since the rest of the code was utterly awful, that the captcha value would be hard coded, and I assumed wrong. With a little research into how simple captchas worked, I curled the captcha image, changed my captcha = 1 statement to the number I found. Wrong again. I had to start over from the initial curl, but this time with a blank cookies.txt

file to trick the website into thinking I was running in an authenticated session. This finally worked, and I was able to curl the landing page after login, piping it into `index_page.html`. From here, I had a bunch of new endpoints to try, so I curled all of them into their own html files, which makes up the rest of my repo. Most of them looked like fairly standard html webpages, with proper boilerplate, and simple click through functionality to other pages. However, on the page stored in `withdraw.html`, the true phishing attempt was found. It turned out to be a run-of-the-mill bank account credential stealer. By tricking the victim into entering their personal details so that they could “withdraw” the bitcoin, the attacker would gain access to their personal bank accounts. Now this is where my reverse-engineering hit a dead end, as the rest of the endpoints were just more fake pages to make the site look legitimate.

Now though, who is the attacker in this case? Many of the code comments were in Chinese (I edited many of the compliments after translating them to improve readability and understanding), and the site appeared to be hosted through a PHP provider ending in `.cn`. Clearly this phishing attack was being carried out from China, but where in China? Running a curl command with DNS tacked on again, plus some grep with regex, we can see that the associated IP is 45.194.36.98. Using a free online IP information lookup, we can see that this IP is registered to a company in Hong Kong called Cloud Innovation Support. Upon googling this business, no results turn up. However, their IP stores administrative data as well, such as a contact email for the service they are hosted through, and that service's mailing address. The domain for the emails is not a visitable page, however researching Ebene, MU, and the term Afrinic and Cloud Innovation returns a company in Seychelles that works directly with a company in Hong Kong called LARUS Ltd. I am unsure of whether the attack was served from the company itself or just passed through their server services, as either is possible. From this I

was able to extract contact information, a street address, and other manners of OSINT.

Now for a more fun section, there were some incredibly bad coding practices that led me to being able to do this. Firstly, it is very suspicious when the login page doesn't run an SQL server query, but instead compares the username to a hardcoded string. Same goes for the password. And the account balance, and every single other piece of information on the website that should have been a database link. This is the equivalent of the old west Hollywood facades, but in website form. This entire site was a spaghetti-coded shell of a website, designed to prey on the people who may not have the greatest tech literacy.

The next steps in this process to remove this attack from circulation would be to report it to the FTC. It is unfortunately not recommended to say, DDoS their servers, or read their traffic, or all other manners of federal crimes. While white-hat hacking/offensive security is typically done with best intentions in mind, there are still laws around all of these protocols. To the people reading this write up, take this as a call to action to not fall for obvious things like this, and if you have the skills, safely, and I mean safely, see what you can find out.